



Hidden Markov Models Explained with a Real Life Example and code

Carolina Bento

Nov 5, 2023 17 min read

LATEST

EDITOR'S PICKS

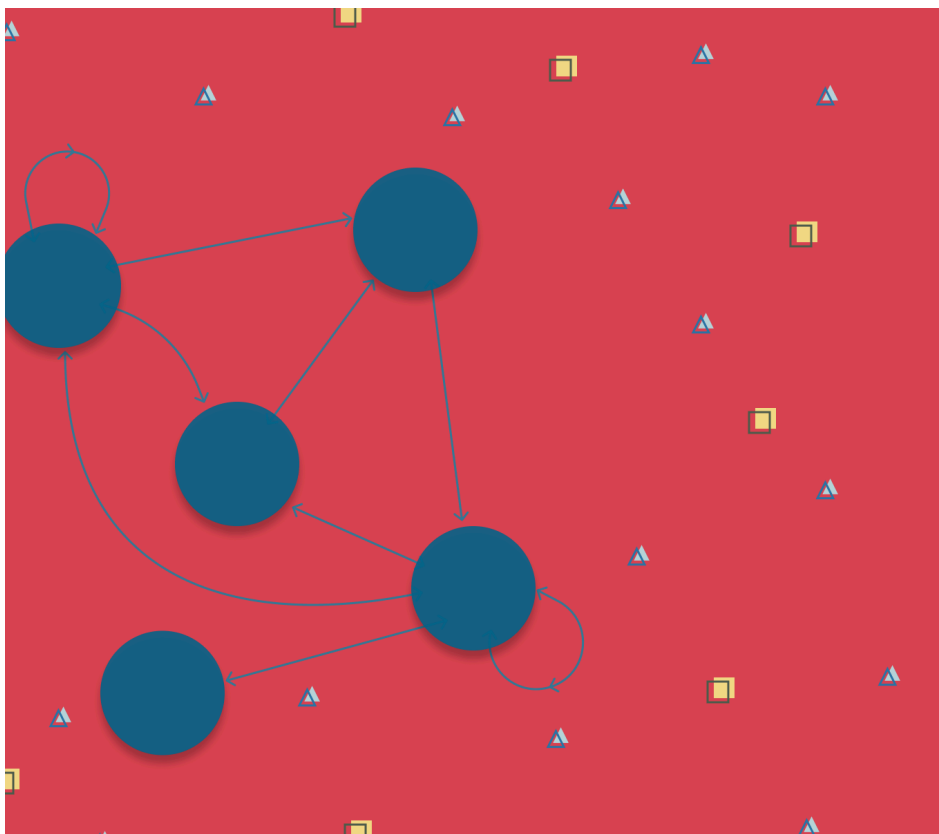
DEEP DIVES

CONTRIBUTE

NEWSLETTER

Sign in

Contributor Portal



Hidden Markov Models are probabilistic models used to solve a wide range of real life problems ranging from something everyone can relate to — how is the weather going to be like next week — to hard molecular biology problems, such as predicting how a protein binds to the human MHC class II molecule[2].

Hidden Markov Models are close relatives of Markov Chains, but their hidden states make them a unique tool to use when you're interested in determining the probability of a sequence of random variables.

In this article we'll breakdown Hidden Markov Models into all its different components and see, step by step with both the Math and Python code, which emotional states led to your training exam. We'll be using the Viterbi Algorithm to determine the probability of observing a specific sequence of outcomes, how you can use the Forward Algorithm to determine the probability of an observed sequence, when you're given a set of hidden states.

LATEST

EDITOR'S PICKS

DEEP DIVES

CONTRIBUTE

NEWSLETTER

Sign in

Contributor Portal

The real world is full of phenomena for which we can't observe the underlying outcome, but can't actually observe the underlying factors that generated those outcomes. One example is predicting the weather: determining if it's going to be rainy or sunny tomorrow based on past weather observations and the observed probabilities of different weather outcomes.

Although driven by factors we can't observe, with a Hidden Markov **Model** it's possible to model these phenomena as a sequence of hidden states and systems.

Hidden Markov Models with Hidden States

Hidden Markov Models, known as HMM for short, are a class of probabilistic models that work as a sequence of labeling problems. They are used to solve the types of problems that describe the evolution of a system over time.

events, which themselves, are dependent on internal factors that can't be directly observed — they are **hidden**[3].

An Hidden Markov Model is made of two distinct stochastic processes, meaning those are processes that can be defined as sequences of random variables — variables that depend on random events.

There's an **invisible process** and an **observable process**.

The **invisible process** is a Markov Chain, like characterizing multiple **hidden states** that are traversed over time to reach an outcome. This is a probabilistic process where the parameters of the Markov Chain, as well as the parameters of the sequence, are in fact probabilities[4].

Hidden Markov Models describe the evolution of a system where events, which themselves, are dependent on internal factors that can't be directly observed — they are **hidden**[3]

Just like any other Markov Chain, in order to know what you're going next, the only thing that matters is the current state — in which state of the Markov Chain you're currently in. Unlike the previous *history* of states you've been in, the current state is what you need to understand where you're going next.

This kind of *short-term* memory is one of the key features of HMMs and it's called the **Markov Assumption**, in which the probability of reaching the next state is only dependent on the probability of the current state.

LATEST

EDITOR'S PICKS

DEEP DIVES

CONTRIBUTE

NEWSLETTER

Sign in

Contributor Portal

$$P(\underbrace{q_i = a \mid q_1 \dots q_{i-1}}_{\text{history of states traversed}}) = P(\overbrace{q_i = a}^{\text{next state}} \mid q_{i-1})$$

Markov Assumption. (Image by Author)

The other key characteristic of an HMM, is that in each observation is only dependent on the state therefore, being completely independent from a chain[5].

The **Markov Assumption** states that the probability the next state is only dependent on the probability state.

This is all great background information on HMM of problems are they actually used in?

HMMs help model the behavior of phenomena. E and allowing to run simulations, you can also ask questions those phenomena:

- **Likelihood** or **Scoring**, as in, determining the observing a sequence
- **Decoding** the best sequence of states that generated observation
- **Learning** the parameters of the HMM that learned given sequence, that traversed a specific sequence

LATEST

EDITOR'S PICKS

DEEP DIVES

CONTRIBUTE

NEWSLETTER

Sign in

Contributor Portal

Let's see this in practice!

Dog Training Performance as an HMM

Today you're not as worried about the weather forecast, what's on your mind is that your dog is possibly graduating from their training lessons. After all the time, effort and dog treats you want is for them to succeed.

During dog training sessions, your four-legged friend does a few actions or tricks, so the trainer can observe their performance. After combining the scores of all the sessions, the trainer determines if your dog graduates or needs additional training. The trainer only sees the outcome, but there are many factors involved that can't be directly observed such as if your dog is happy, if they don't like the trainer at all or the conditions of the environment.

None of these are directly observed, unless there is a specific action your dog does only when they feel a certain way. Would be great if they could express how they feel about the training in the future!

With Hidden Markov Models fresh in your mind, this is a perfect opportunity to try to predict how your dog will perform during the exam. They might get a certain score because they were feeling tired, maybe they were hungry, or they were nervous around the trainer.

Your dog has been taking lessons for a while and you have collected data during that training, you have all the building blocks needed to build a Hidden Markov Model.

LATEST

EDITOR'S PICKS

DEEP DIVES

CONTRIBUTE

NEWSLETTER

Sign in

Contributor Portal

In order to build a HMM that models the performance of your dog in the training evaluation you need:

- Hidden States
- Transition Matrix
- Sequence of Observations
- Observation Likelihood Matrix
- Initial Probability Distribution

Hidden States are those non-observable factors observation sequence. You'll only consider if you Happy.

$$Q = Q_1, Q_2, \dots, Q_n$$

$$Q = \text{Tired, Happy}$$

Different hidden states in the HMM. (Image by Author)

Knowing your dog very well, the non-observable impact their exam performance are simply being

Next you need to know what's the probability of state to another, which is captured in a **Transiti** matrix must also be **row stochastic** meaning the from one state to any other state in the chain, e matrix, must sum to one.

LATEST

EDITOR'S PICKS

DEEP DIVES

CONTRIBUTE

NEWSLETTER

Sign in

Contributor Portal

$$A = a_{11}, a_{ij}, \dots, a_{NM} \quad \sum_{j=1}^N a_{ij} = 1 \forall i$$

Sum of state transitions = 1

LATEST

EDITOR'S PICKS

DEEP DIVES

CONTRIBUTE

NEWSLETTER

Sign in

Contributor Portal

	Tired	Happy
Tired	0.4	0.6
Happy	0.2	0.8

Transition Matrix: represents the probability of moving from one state to

Regardless of what type of problem you're solving, you need a **Sequence of Observations**. Each observation is the result of traversing the Markov Chain. Each observation is from a specific vocabulary.

$$v = v_1, v_2, \dots, v_n$$

Performance = $\begin{cases} \text{Fail} \\ \text{OK} \\ \text{Perfect} \end{cases}$

[LATEST](#)
[EDITOR'S PICKS](#)
[DEEP DIVES](#)
[CONTRIBUTE](#)

Vocabulary (Image by Author)

In the case of your dog’s exam you observe the each trial, which can be *Fail*, *OK* or *Perfect*. These possible *terms* in the observation vocabulary.

You also need the **Observation Likelihood Matrix** probability of an observation being generated from

	Fail	OK
Tired	0.3	0.5
Happy	0.1	0.5

Observation Likelihood Matrix. (Image by Author)

Finally, there’s the **Initial Probability Distribution**. This is the probability that the Markov Chain will start in each specific hidden state.

There can also be some states will never be the starting state in the Markov Chain. In these situations, their initial probability is zero. And just like the probabilities in the Transition Matrix, these sum of all initial probabilities must add up to one.

$$\pi = \pi_1, \pi_2, \pi_3, \dots, \pi_n$$

$$\pi = [\text{Tired}, \text{Happy}] =$$

Initial Probabilities (Image by Author)

The Initial Probability Distribution, along with the Transition Matrix and the Observation Likelihood, make up the **probabilistic Hidden Markov Model (HMM)**. These are the probabilities you’re figuring out to learn which specific HMM could have generated the sequence of observations and hidden states, and to *learn* which specific HMM could have generated the sequence.

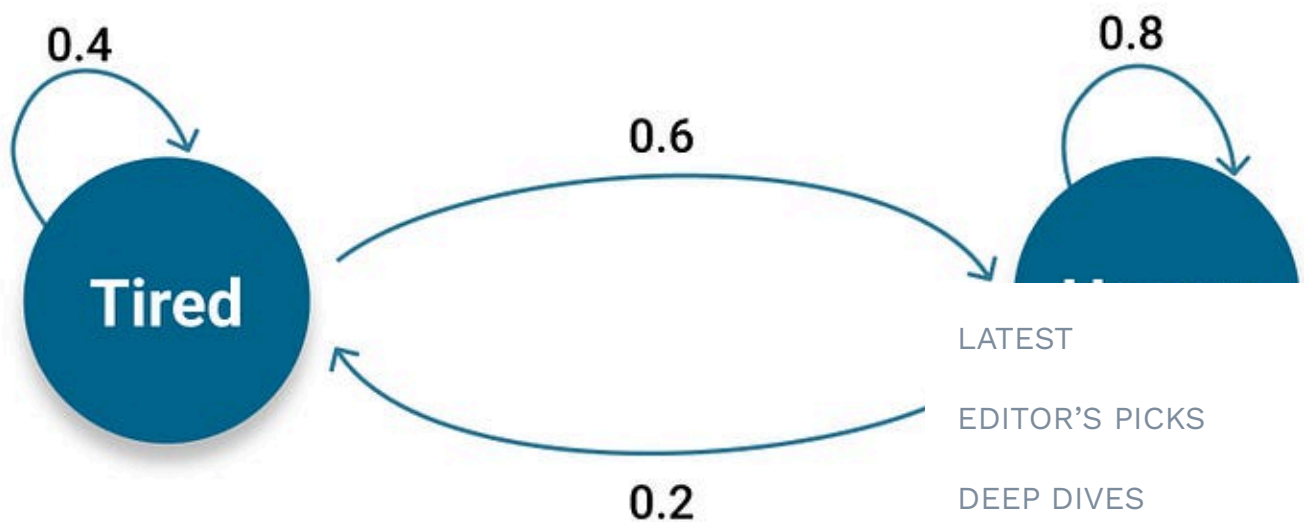
Putting all of these pieces together, this is what the final HMM model that represents your dog’s performance could look like

- LATEST
- EDITOR’S PICKS
- DEEP DIVES
- CONTRIBUTE

NEWSLETTER

Sign in

Contributor Portal



Hidden states and the transition probabilities between them.

During the exam, your dog will perform three trials. Your dog will only perform a trial if they don't *Fail* in two of those trials.

At the end of the day, if your dog needs more trials, they will perform them all the same. The big question circling you is: *What are they feeling during the exam?*

Imagine a scenario where they graduate with a *Perfect* exactly in this order, what sequence of states will they be in? Will they be mostly tired, or hungry, or maybe a mix of both?

This type of problem falls right under the category of *Decoding* problems that HMMs can be applied to. If you're interested in figuring out what's the best sequence of states that generated a specific sequence of observations, this is the *Perfect*.

The problem of decoding the sequence of states given a sequence of observations leverages the **Viterbi** algorithm.

LATEST

EDITOR'S PICKS

DEEP DIVES

CONTRIBUTE

NEWSLETTER

Sign in

Contributor Portal

However, is worth doing a short detour and take a peek into how you could calculate the probability of a given observation sequence — a Likelihood task — using the **Forward Algorithm**. This will set the stage to better understanding how the Viterbi Algorithm works.

The Forward Algorithm

If you were modeling this problem like a regular Markov Chain, where you wanted to calculate the likelihood of observing a sequence of outcomes *OK, Fail, Perfect* you'd traverse the chain by considering each specific state that generates the desired outcome. At each step you would take the conditional probability of the current outcome given that you've observed the previous outcomes and multiply that probability by the transition probability from one state to the other.

The big difference is that, in a regular Markov Chain, the states are well known and observable. Not in an Hidden Markov Model. In a Hidden Markov Model you observe a sequence of outcomes without knowing which specific sequence of hidden states generated that. You traversed in order to observe that.

The big difference is that, in a regular Markov Chain, the states are well known and observable. Not in an Hidden Markov Model.

At this point you might be thinking, *Well I can simulate all possible paths and eventually have a rule to pick the most equivalent paths*. The mathematical definition for the Forward Algorithm looks something like this

[LATEST](#)

[EDITOR'S PICKS](#)

[DEEP DIVES](#)

[CONTRIBUTE](#)

[NEWSLETTER](#)

[Sign in](#)

[Contributor Portal](#)

joint probability of observing
a particular hidden state
sequence

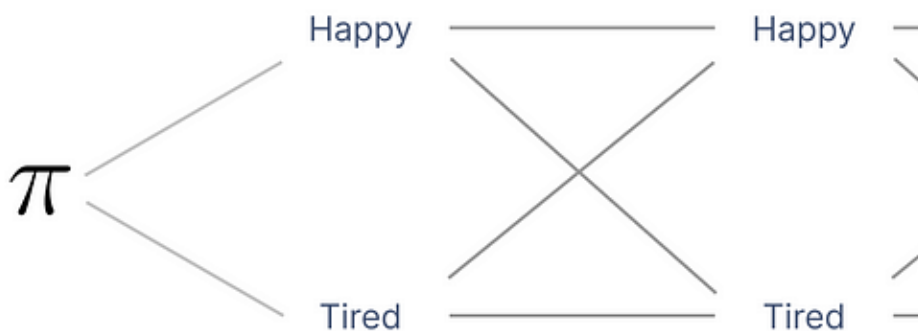
$$\underbrace{P(O)}_{\substack{\text{observed sequence} \\ \text{of scores}}} = \sum_Q \overbrace{P(O, Q)}^{\text{joint probability of observing a particular hidden state sequence}} = \sum_Q P(O|Q) \times P(Q)$$

hidden state
sequence

Calculating the probability of observing a sequence of outcomes, traversing every possible hidden state sequence.
(Image by Author)

That's one strategy for sure! You'd have to calculate the probability of observing the sequence *OK, Fail, Perfect* for every possible combination of hidden states that could ever generate that sequence.

When you have a small enough number of hidden states, it's possible to calculate the probability of observing a sequence of outcomes, it's possible to calculate the probability of observing a sequence of outcomes within a reasonable time.



Outline of the possible paths in your HMM (Image by Author)

LATEST

$P(O|Q)$

For M hidden states

EDITOR'S PICKS

DEEP DIVES

CONTRIBUTE

NEWSLETTER

Sign in

Contributor Portal

Thankfully, the Hidden Markov model you just defined is relatively simple, with 3 observed outcomes and 2 hidden states.

For an observed sequence of length L outcomes, on a HMM with M hidden states, you have “ M to the power L ” possible states which in your case, means *2 to the power of 3*, i.e., 8 possible paths for the sequence *OK — Fail — Perfect*, involving an exponential

computational complexity of $O(M^L L)$, describe

As the complexity of the model increases, the need to take into account grows exponentially.

As the complexity of the model increases, the you need to take into account grows exponen

This is where the **Forward Algorithm** shines.

The Forward Algorithm calculates the probability of the observed sequence, without the need to calculate probabilities of all possible paths that form that

Instead of computing the probabilities of all possible paths that form that sequence the algorithm defines the **forward variable** f_t and calculates its value recursively.

LATEST

EDITOR'S PICKS

DEEP DIVES

CONTRIBUTE

NEWSLETTER

Sign in

Contributor Portal

$$\alpha(n, i) = P(x, \dots, x_n, y_n = i | \Theta)$$

nth outcome in the sequence of observed outcomes x
Current Hidden State
Hidden State i in the sequence of hidden states y
HMM parameters

$$\alpha(n, i) = \sum_k [\alpha(n-1, k) t(k, i) P(x_n = i | y_n = i, \Theta)]$$

previous step in recursion
Transition probability from k to i
LATEST

Pro EDITOR'S PICKS

DEEP DIVES

CONTRIBUTE

How the forward variable is calculated recursively. (Image)

NEWSLETTER

Sign in

Contributor Portal

The fact that it uses recursion, is the key reason it is faster than calculating all the probabilities of fact, it can calculate the probability of observing only “L times M squared” computations, instead of L times L”.

In your case, with 2 hidden states and a sequence of outcomes, it's the difference between calculating $O(M^L L) = 2^3 \times 3 = 8 \times 3 = 24$ times, as opposed to $O(L^2 L) = 12$ times.

This reduction in the number of calculations is achieved by Dynamic Programming, a programming technique that uses auxiliary data structures to store intermediate results, therefore making sure the same calculations are not performed multiple times.

Every time the algorithm is about to calculate a value, it checks if it has already computed it, and if so, it retrieves that value in the intermediate data structure. Otherwise, the probability is calculated and the value is stored.

Let's get back to your decoding problem, using the Viterbi Algorithm.

The Viterbi Algorithm

Thinking in *pseudo code*, If you were to brute force your way into decoding the sequence of hidden states that generate the observation sequence, all you needed to do was

- generate all possible permutations of paths that generate the desired observation sequence
- use the Forward Algorithm to calculate the probability of the observation sequence, for each possible sequence of hidden states
- pick the sequence of hidden states with the highest probability

[LATEST](#)

[EDITOR'S PICKS](#)

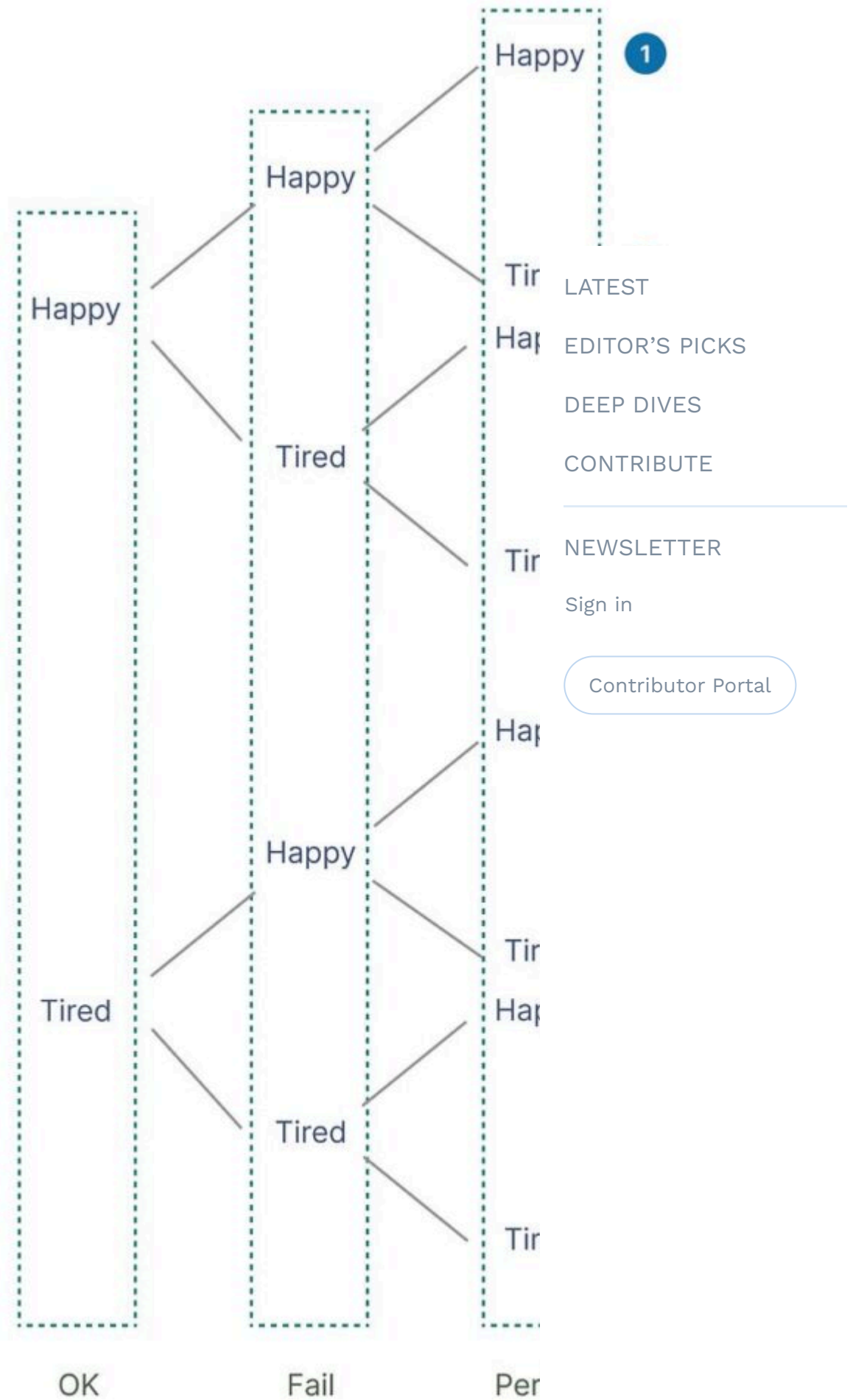
[DEEP DIVES](#)

[CONTRIBUTE](#)

[NEWSLETTER](#)

[Sign in](#)

[Contributor Portal](#)



For your specific HMM, there are 8 possible paths that lead to an outcome of *OK — Fail — Perfect*. Add just one more observation, and you'll have double the amount of possible sequences of hidden states! Similarly to what was described for the Forward Algorithm, you easily end up with an exponentially complex performance ceiling.

The Viterbi Algorithm, gives you a hand with that.

When the sequence of hidden states in the HMM is given, each step, the probability $v_t(j)$ is the probability of the hidden state j after seeing the observation sequence traversed through the most probable state that

[LATEST](#)

[EDITOR'S PICKS](#)

[DEEP DIVES](#)

[CONTRIBUTE](#)

[NEWSLETTER](#)

[Sign in](#)

[Contributor Portal](#)

$$v_t(j) = \max_{q_1, q_2, \dots, q_t} P(q_1 q_2 \dots q_{t-1}, o_1 o_2 \dots o_t | \lambda)$$

$v_t(j)$: Viterbi path to hidden state j
 $\max_{q_1, q_2, \dots, q_t}$: indicates the algorithm is looking for the most probable path
 $P(q_1 q_2 \dots q_{t-1}, o_1 o_2 \dots o_t | \lambda)$: hidden path sequence until the last time step, observation sequence until the last time step

Viterbi path to hidden state j on time step t . (Image by Author)

The key to decoding the sequence of hidden states for a specific observation sequence, is this concept of a **path**. Also called the **Viterbi path**, the most probable path that has highest likelihood, from all the paths that lead to any given hidden state.

The key to decoding the sequence of hidden states for a specific observation sequence, is to use the most probable path that leads to any given hidden state.

You can draw a parallel between the Forward Algorithm and the Viterbi Algorithm. Where the Forward Algorithm sums all probabilities to obtain the likelihood of reaching a certain state taking into account all the paths that lead there, the Viterbi algorithm doesn't want to explore all possibilities. It focuses on the most probable path that leads to any given state.

Going back to the task of decoding the sequence that lead to the scores of OK — Fail — Perfect in exam, *running* the **Viterbi Algorithm** by hand would

[LATEST](#)

[EDITOR'S PICKS](#)

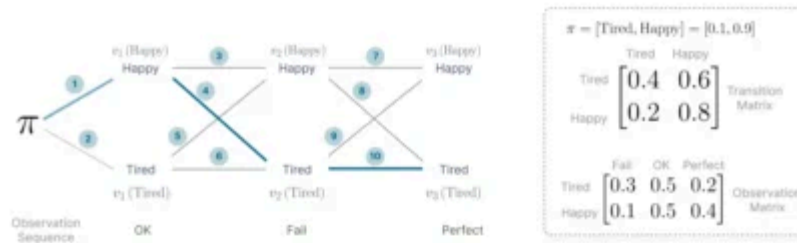
[DEEP DIVES](#)

[CONTRIBUTE](#)

[NEWSLETTER](#)

[Sign in](#)

[Contributor Portal](#)



$$1 \rightarrow P(\text{Happy} | \text{Start}) \times P(\text{OK} | \text{Happy}) = 0.9 \times 0.5 = 0.45$$

$$v_1(\text{Happy}) = \text{1} = P(\text{Happy} | \text{Start}) \times P(\text{OK} | \text{Happy}) = 0.9 \times 0.5 = 0.45$$

$$2 \rightarrow P(\text{Tired} | \text{Start}) \times P(\text{OK} | \text{Tired}) = 0.1 \times 0.5 = 0.05$$

$$v_1(\text{Tired}) = \text{2} = P(\text{Tired} | \text{Start}) \times P(\text{OK} | \text{Tired}) = 0.1 \times 0.5 = 0.05$$

$$3 \rightarrow P(\text{Happy} | \text{Happy}) \times P(\text{Fail} | \text{Happy}) = 0.8 \times 0.1 = 0.08$$

$$4 \rightarrow P(\text{Tired} | \text{Happy}) \times P(\text{Fail} | \text{Tired}) = 0.2 \times 0.3 = 0.06$$

$$5 \rightarrow P(\text{Happy} | \text{Tired}) \times P(\text{Fail} | \text{Happy}) = 0.6 \times 0.1 = 0.06$$

$$6 \rightarrow P(\text{Tired} | \text{Tired}) \times P(\text{Fail} | \text{Tired}) = 0.4 \times 0.4 = 0.16$$

$$v_2(\text{Happy}) = \max((v_1(\text{Happy}) \times \text{3}), (v_1(\text{Tired}) \times \text{5})) =$$

$$= \max((0.45 \times 0.08), (0.05 \times 0.06)) =$$

$$= \max(0.036, 0.003) = 0.036$$

$$v_2(\text{Tired}) = \max((v_1(\text{Happy}) \times \text{4}), (v_1(\text{Tired}) \times \text{6})) =$$

$$= \max((0.45 \times 0.06), (0.05 \times 0.12)) =$$

$$= \max(0.027, 0.006) = 0.027$$

$$7 \rightarrow P(\text{Happy} | \text{Happy}) \times P(\text{Perfect} | \text{Happy}) = 0.8 \times 0.4 = 0.32$$

$$8 \rightarrow P(\text{Tired} | \text{Happy}) \times P(\text{Perfect} | \text{Tired}) = 0.2 \times 0.2 = 0.04$$

$$9 \rightarrow P(\text{Happy} | \text{Tired}) \times P(\text{Perfect} | \text{Happy}) = 0.6 \times 0.4 = 0.24$$

$$10 \rightarrow P(\text{Tired} | \text{Tired}) \times P(\text{Perfect} | \text{Tired}) = 0.4 \times 0.2 = 0.08$$

$$v_3(\text{Happy}) = \max((v_2(\text{Happy}) \times \text{7}), (v_2(\text{Tired}) \times \text{9})) =$$

$$= \max((0.36 \times 0.32), (0.027 \times 0.24)) =$$

$$= \max(0.012, 0.006) = 0.012$$

$$v_3(\text{Tired}) = \max((v_2(\text{Happy}) \times \text{8}), (v_2(\text{Tired}) \times \text{10})) =$$

$$= \max((0.36 \times 0.04), (0.027 \times 0.08)) =$$

$$= \max(0.001, 0.002) = 0.002$$

Decode Sequence using Backpointers

$$\text{Decoded Sequence} = \max(v_3(\text{Happy}), v_3(\text{Tired}))$$

$$= v_3(\text{Tired}) \rightarrow v_2(\text{Tired}) \rightarrow v_1(\text{Happy})$$

$$\text{10} \rightarrow \text{4} \rightarrow \text{1}$$

LATEST

EDITOR'S PICKS

DEEP DIVES

CONTRIBUTE

NEWSLETTER

Sign in

Contributor Portal

Another unique characteristic of the Viterbi algorithm is that it must have a way to keep track of all the paths that led to any given hidden state, in order to compare their probabilities. To do that it keeps track of **backpointers** to each hidden state, using an auxiliary data structure typical of dynamic programming algorithms. That way it can easily access the probability of any viterbi path traversed in the past.

Backpointers are the key to figure out the most leads to an observation sequence.

In the example of your dogs' exam, when you calculate the probabilities for paths $v_3(\text{Happy})$ and $v_3(\text{Tired})$, you pick the path with the highest probability and start going backwards, i.e., backtracking the paths that led to where you are.

LATEST

EDITOR'S PICKS

DEEP DIVES

CONTRIBUTE

NEWSLETTER

Sign in

Contributor Portal

The Viterbi Algorithm in Python

Doing all of this by hand is time consuming and error-prone. If you have a single significant digit and you might have to start re-check all your probabilities!

The good news is that you can leverage software like [hmmlearn](#), and with a few lines of code you can find the sequence of hidden states that lead to your dog's behavior — *Fail* — *Perfect* in the trials, exactly in this order:

```
from hmmlearn import hmm<br>import numpy as np<br><br>## Parameters for the HMM<br>with specific parameters and simulating the exam<br>print('Parameters used for the HMM')<br># init_params are the parameters used to initialize the HMM<br># s -> start probability<br># t -> transition probabilities<br># e -> emission probabilities<br>model = hmm.CategoricalHMM(n_components=2, n_states=2, n_symbols=2,<br>random_state=425, init_params='ste')<br><br># initial probabilities<br># initial probability of starting in the Tired state = 0<br># initial probability of starting in the Happy state = 1<br>initial_distribution = np.array([0., 1.])
```

```

model.startprob_ = initial_distribution<br><br>print("Step 1. Complete -
Defined Initial Distribution")<br><br># transition probabilities<br>#
tired    happy<br># tired    0.4        0.6<br># happy    0.2        0.8<br>
<br>transition_distribution = np.array([[0.4, 0.6], [0.2, 0.8]])
<br>model.transmat_ = transition_distribution<br>print("Step 2. Complete -
Defined Transition Matrix")<br><br># observation probabilities<br>#          Fail
OK        Perfect<br># tired    0.3    0.5        0.2<br># happy    0.1    0.5
0.4<br><br>observation_probability_matrix = np.array([[0.3, 0.5, 0.2], [0.1, 0.5,
0.4]])<br>model.emissionprob_ = observation_probability_matrix<br>print("Step 3.
Complete - Defined Observation Probability Matrix")<br><br># simulate performing
100,000 trials, i.e., aptitude tests<br>trials, simulated_
model.sample(100000)<br><br># Output a sample of the simul
Fail<br># 1 -> OK<br># 2 -> Perfect<br>print("\nSample of :
Based on Model Parameters")<br>print(trials[:10])<br><br>##
hidden state sequence that leads<br>## to an observation se
Perfect<br><br># split our data into training and test sets
<br>X_train = trials[:trials.shape[0] // 2]<br>X_test = tr
2:]<br><br>model.fit(X_train)<br><br># the exam had 3 tria
following score: OK, Fail, Perfect (1, 0 , 2)<br>exam_obs
<br>predicted_states = model.predict(X=[[1, 0, 2]])<br>pri
State Transitions that were being the exam scores OK, Fail,
Tired , "<br>          "1 -> Happy"<br>print(predicted_states'

```

[LATEST](#)
[EDITOR'S PICKS](#)
[DEEP DIVES](#)
[CONTRIBUTE](#)
[NEWSLETTER](#)
[Sign in](#)
[Contributor Portal](#)

In a few seconds you get an output that matches the calculations you did by hand, much faster and with less error.

```

Setup HMM model with parameters
Step 1. Complete - Defined Initial Distribution
Step 2. Complete - Defined Transition Matrix
Step 3. Complete - Defined Observation Probability Matrix

Sample of Simulated Trials - Based on Model Parameters
[[1]
 [1]
 [2]
 [1]
 [2]
 [0]
 [1]
 [1]
 [1]
 [2]]
Even though the 'startprob_' attribute is set, it will be overwritten during initialization
Even though the 'transmat_' attribute is set, it will be overwritten during initialization
Even though the 'emissionprob_' attribute is set, it will be overwritten during initialization
Predict the Hidden State Transitions that were being the exam scores OK, Fail, Perfect:
0 -> Tired , 1 -> Happy
[1 0 0]

```

Output of running the code above. (Image by Autl)

Conclusion

What's fascinating about Hidden Markov Models is how this statistical tool created in the mid 1960's [6] is so powerful and applicable to real world problems in such distinct areas, from weather forecasting to finding the next word in a sentence.

In this article, you had the chance to learn about the different components of an HMM, how they can be applied to different types of tasks, and spotting the similarities between the Forward Algorithm and Viterbi Algorithm. Two very similar algorithms that use dynamic programming to deal with the exponential calculations involved.

Either doing the calculations by hand or plugging them into TensorFlow code, hope you enjoyed diving into the world of HMMs.

Thank you for reading!

[LATEST](#)

[EDITOR'S PICKS](#)

[DEEP DIVES](#)

[CONTRIBUTE](#)

[NEWSLETTER](#)

[Sign in](#)

[Contributor Portal](#)

References

1. D. Khiatani and U. Ghose, "Weather forecast using Hidden Markov Model," 2017 International Conference on Smart Computing and Communication Technologies for Smart Infrastructure, Gurgaon, India, 2017, pp. 220–225, doi: 10.1109/IC3TSN.2017.8284480.
2. Noguchi H, Kato R, Hanai T, Matsubara Y, Horiuchi T, Kobayashi T. Hidden Markov model-based prediction of antigenic peptides that interact with MHC class II. *Biosci Bioeng*. 2002;94(3):264–70. doi: 10.1266/bse.94.264.
3. Yoon BJ. Hidden Markov Models and their Applications in Biological Sequence Analysis. *Curr Genomics*. 2018;19(1):1–10.

Sep;10(6):402–15. doi: 10.2174/138920209789177575. PMID: 20190955; PMCID: PMC2766791.

4. Eddy, S. What is a hidden Markov model?. *Nat Biotechnol* **22**, 1315–1316 (2004). <https://doi.org/10.1038/nbt1004-1315>
5. Jurafsky, Dan and Martin, James H.. *Speech and language processing : an introduction to natural language processing, computational linguistics, and speech recognition*. LATEST
River, N.J.: Pearson Prentice Hall, 2009. EDITOR'S PICKS
6. Baum, Leonard E., and Ted Petrie. “Statistical Probabilistic Functions of Finite State Markov Processes”. *Annals of Mathematical Statistics* 37, no. 6 (1965): 1554–1562. DEEP DIVES
CONTRIBUTE

NEWSLETTER

Sign in

• • •

Contributor Portal

WRITTEN BY

Carolina Bento

See all from Carolina Bento

Topics:

Data Science

Editors Pick

Hidden Mark Models

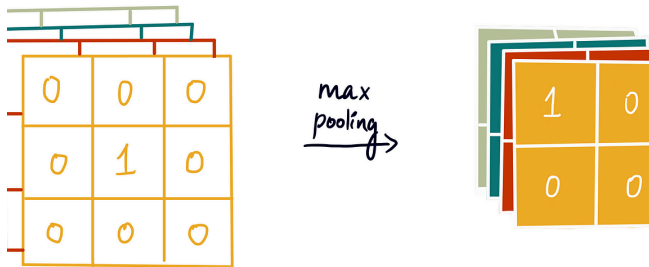
Machine Learning

Statistics

Share this article:



Related Articles



ARTIFICIAL INTELLIGENCE

Implementing Convolutional Neural Networks in TensorFlow

Step-by-step code guide to building a Convolutional Neural Network

Shreya Rao

August 20, 2024 6 min read



ARTIFICIAL INTELLIGENCE

How to Forecast Hierarchical Time Series

A beginner's guide to forecast reconciliation

Dr. Robert Kübler

August 20, 2024 13 min read

ARTIFICIAL INTELLIGENCE

What Do Large Language Models “Understand”

LATEST

A deep dive on understanding LLMs

Tarik Dzekman

August 21, 2024

EDITOR'S PICKS

DEEP DIVES

CONTRIBUTE

NEWSLETTER

Sign in

Contributor Portal



DATA SCIENCE

Hands-on Time Series Detection with Python

Here's how to use Python to detect signals in time series data...

Piero Paialunga

August 21, 2024



MACHINE LEARNING

3 AI Use Cases (That Are Not a Chatbot)

Feature engineering, structuring unstructured data, and lead scoring

Shaw Talebi

August 21, 2024 7 min read



DATA SCIENCE

Back To Basics, Part Uno: Linear Regression and Cost Function

An illustrated guide on essential machine learning concepts

Shreya Rao

February 3, 2023 6 min read

DATA SCIENCE

Solving a Constrained Project Scheduling Problem with Quantum Annealing

Solving the resource constrained project scheduling problem (RCPSP) with D-Wave's hybrid constrained quadratic model (CQM)

Luis Fernando PÉI

August 20, 2024

LATEST

EDITOR'S PICKS

DEEP DIVES

CONTRIBUTE

NEWSLETTER

Sign in

Contributor Portal



Your home for data science and AI. The world’s leading publication for data science, data analytics, data engineering, machine learning, and artificial intelligence professionals.

© Insight Media Group, LLC 2025

Subscribe to Our Newsletter

ABOUT · ADVERTISE · PRIVACY POLICY · TERMS

LATEST

EDITOR’S PICKS

DEEP DIVES

CONTRIBUTE

NEWSLETTER

Sign in

Contributor Portal